

Grafuri

Graful = modelul tipic pentru reprezentarea relațiilor între obiecte de același tip:

- orașe înfrățite,
- colaboratori,

Relațiile dintre obiecte pot fi reflexive sau nu

⇒ **graf neorientat** sau **orientat**.

Obiectele: **nodurile (vârfurile)** grafului

Relațiile: **muchiile (arcele)** care unesc nodurile;

Nodurile, respectiv arcele pot fi etichetate; arcelor le sunt asociate frecvent costuri.

Grafuri

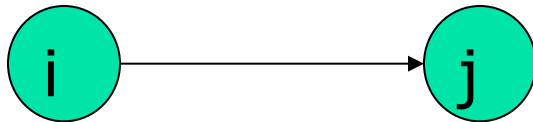
Graf orientat (en.digraph):

$G(V,A)$

unde V – mulțimea vârfurilor (vertices)

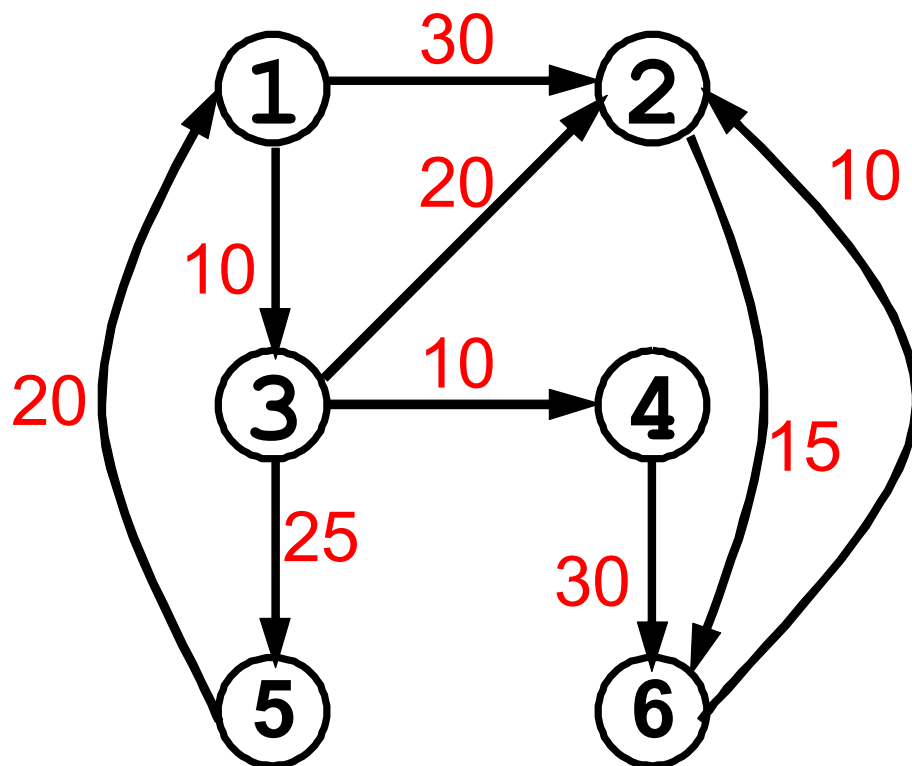
A – mulțimea arcelor (edges)

Arc: pereche ordonată de noduri (sursă, destinație) – stabilește relația succesor/predecesor direct.



Descriere graf orientat

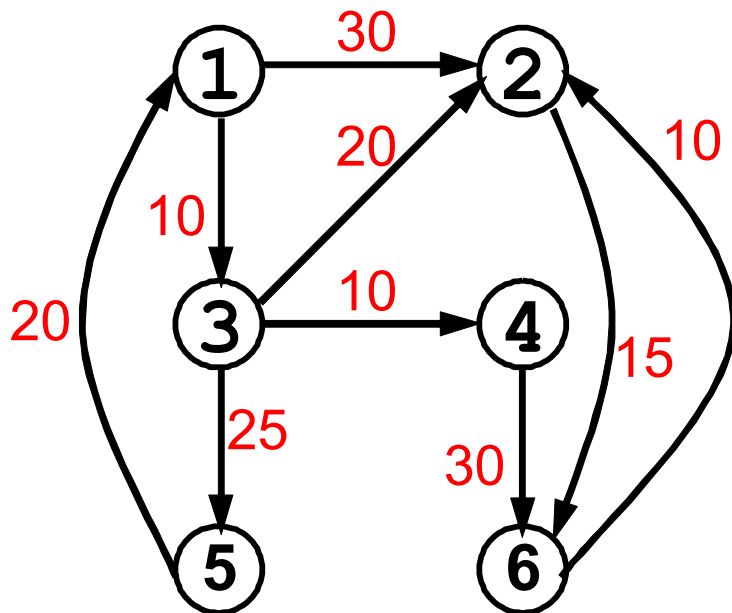
Număr noduri și listă arce



6

1	2	30
1	3	10
2	6	15
3	2	20
3	4	10
3	5	25
4	6	30
5	1	20
6	2	10

Reprezentare graf orientat



a. Matrice de adiacențe (costuri)

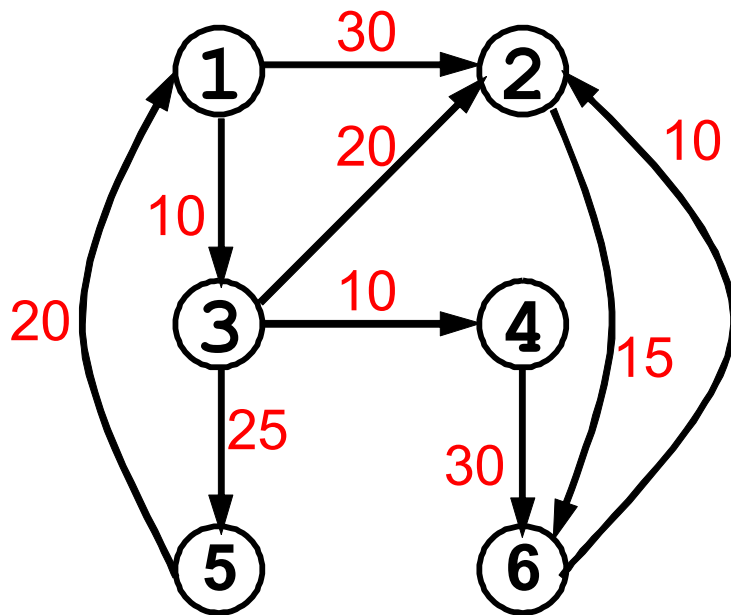
	1	2	3	4	5	6
1	-	30	10	-	-	-
2	-	-	-	-	-	15
3	-	20	-	10	25	-
4	-	-	-	-	-	30
5	20	-	-	-	-	-
6	-	10	-	-	-	-

b. Liste de adiacențe

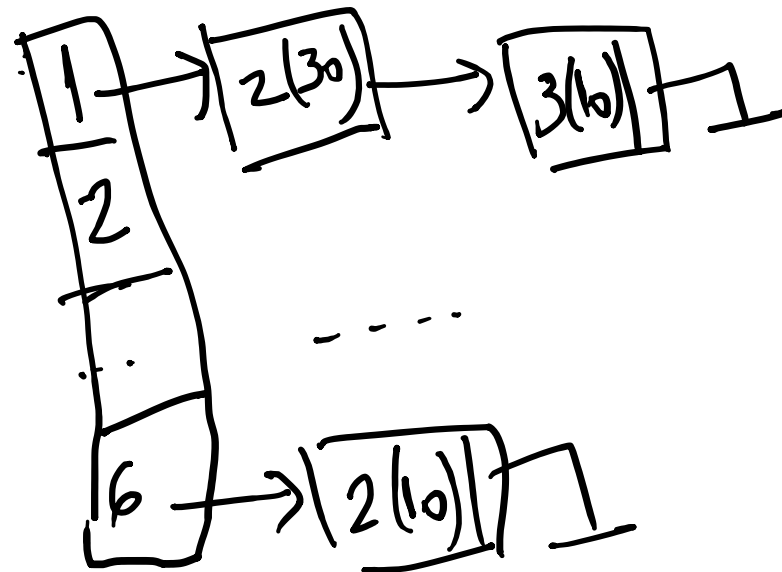
1 : 2 (30) 3 (10)
2 : 6 (15)
3 : 2 (20) 4 (10) 5 (25)
4 : 6 (30)
5 : 1 (20)
6 : 2 (10)

Reprezentare graf orientat

b. Liste de adiacențe

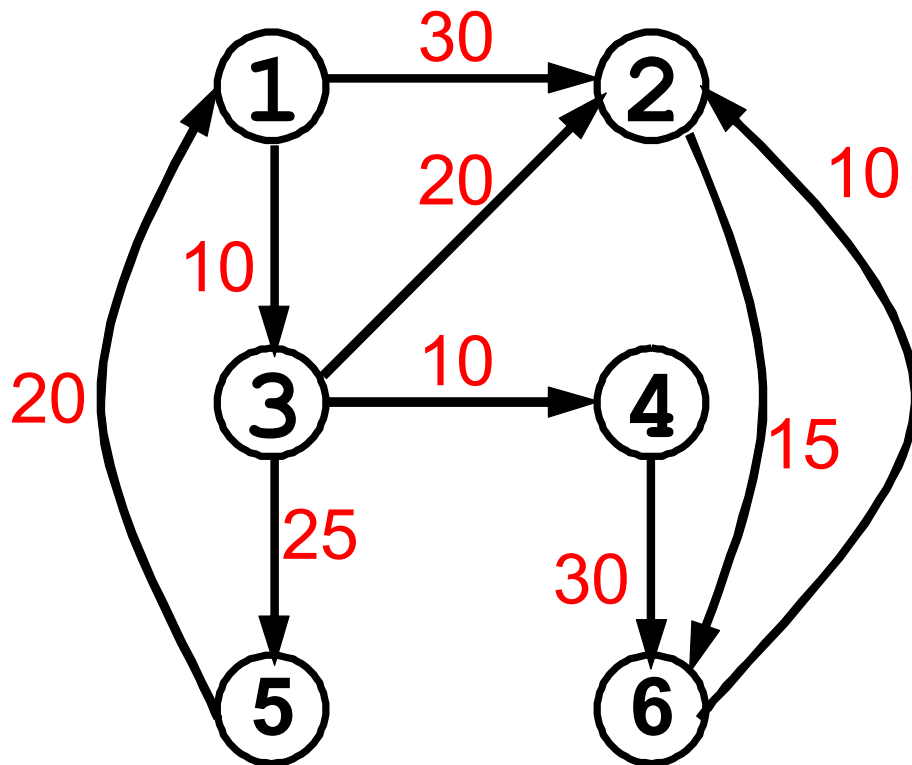


1 : 2 (30) 3 (10)
2 : 6 (15)
3 : 2 (20) 4 (10) 5 (25)
4 : 6 (30)
5 : 1 (20)
6 : 2 (10)



Grafuri

- i-grad/e-grad: număr arce care ajung în /pleacă din nod
- i-grad: gradul de intrare al unui nod
- e-grad: gradul de iesire al unui nod

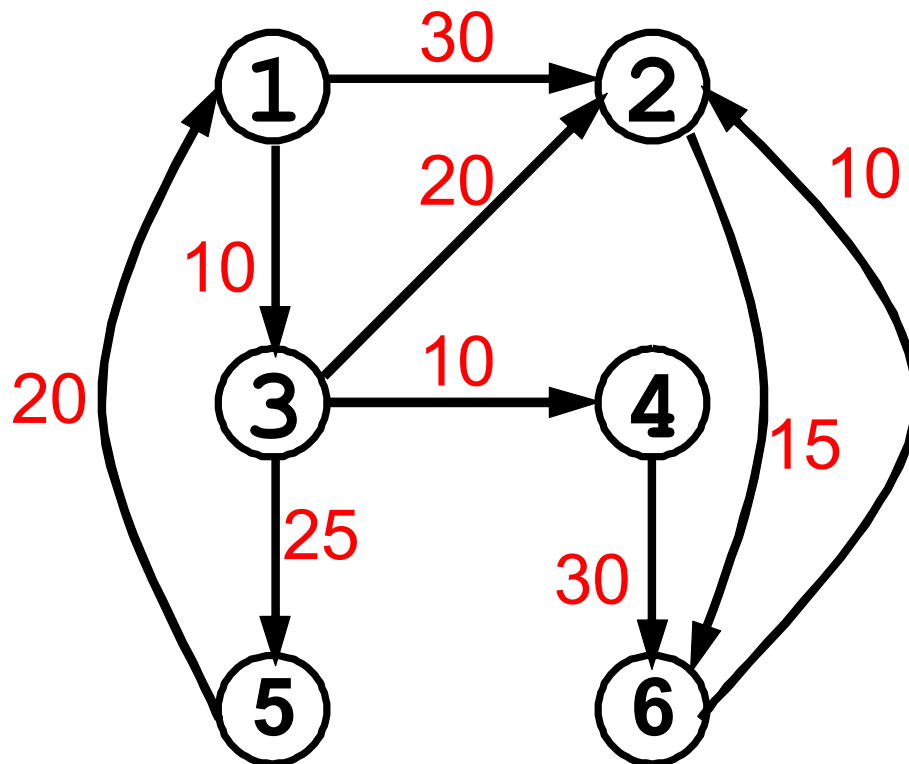


i-grad(2)=(3)
e-grad(2)=1

Noduri
adiacente cu
nodul n:
nodurile
pentru care
existe arce de
la n la v

Grafuri

- **cale**: succesiune de noduri conectate prin arce;
numărul de arce parcurse = lungimea căii
- **cale simplă**: trece numai prin noduri distincte
- **ciclu**: cale pe care primul nod = ultimul nod
- **graf aciclic**: nu conține cicluri.



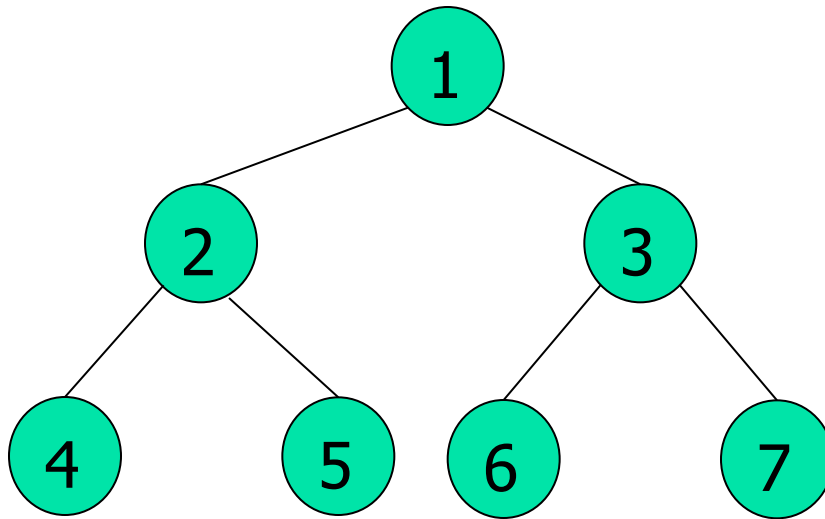
Cale: 1, 2, 6, 2

Cale: 1, 3, 4, 6, 2, 6

Ciclu: 1, 3, 5, 1

Grafuri

Arbori:



Arborii sunt cazuri particulare de grafuri aciclice orientate, cu un singur nod cu $i\text{-grad} = 0$ și restul cu $i\text{-grad} = 1$.

Parcurgere grafuri

- Parcurgere în adâncime (Depth First Search / DFS)
- Parcurgere în lățime (Breadth First Search / BFS)

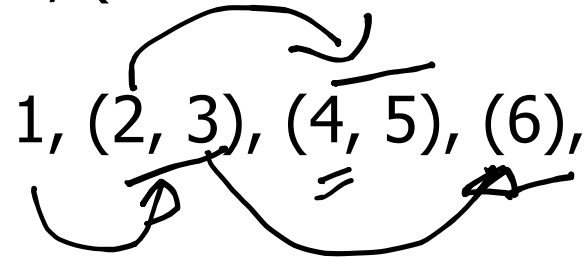
Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1	2	30
1	3	10
2	4	15
2	5	20
3	6	10
4	6	30
5	3	20
5	4	15
6	2	10

1, (noduri adiacente cu 1), ...



Coada: inserare noduri adiacente
vizitat: 0/1 (daca nodul i a fost vizitat)
– in momentul in care se introduce
nodul in coada

1, 2, 3, 4, 5, 6

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:1

vizitat: 0, 0, 0, 0, 0, 0

Coadă:

Afisare 1

Vizitat: 1, 0, 0, 0, 0, 0

Coadă: 2, 3

Coadă:3

Afisare 1, 2

Vizitat: 1, 1, 0, 0, 0, 0

Coadă: 3, 4, 5

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:3

Afisare 1, 2

Vizitat: 1, 1, 0, 0, 0, 0

Coadă: 3, 4, 5

Coadă:4, 5

Afisare 1, 2, 3

Vizitat: 1, 1, 1, 0, 0, 0

Coadă: 4, 5, 6

Coadă: 5, 6

Afisare 1, 2, 3, 4

Vizitat: 1, 1, 1, 1, 0, 0

Coadă: 5, 6

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coadă:1

vizitat: 1, 0, 0, 0, 0, 0

Coadă:

Afisare 1

Vizitat: 1, 1, 1, 0, 0, 0

Coadă: 2, 3

Coadă: 3

Afisare 1, 2

Vizitat: 1, 1, 1, 1, 1, 0

Coadă: 3, 4, 5

Parcurgere în latime

Parcurgere începând de la nodul 1

Descriere graf

1 2 30

1 3 10

2 4 15

2 5 20

3 6 10

4 6 30

5 3 20

5 4 15

6 2 10

Coada: 4, 5

Afisare 1, 2, 3

Vizitat: 1, 1, 1, 1, 1, 1

Coada: 4, 5, 6

Coada: 5, 6

Afisare 1, 2, 3, 4

Vizitat: 1, 1, 1, 1, 1, 1

Coada: 5, 6

Coada: -

Afisare: 1, 2, 3, 4, 5, 6

Vizitat: 1, 1, 1, 1, 1, 1

Parcurgere în latime

Parcurgere în adâncime pornind de la nodul n

`IntrQ(q, n)`

`marcheaza n vizitat`

`while(q != vida)`

`{`

`n = ExtrQ(q)`

`for fiecare a adiacent cu n`

`{`

`daca a e nevizitat`

`{`

`marcheaza a vizitat`

`IntrQ(q, a)`

`}`

`}`

`}`

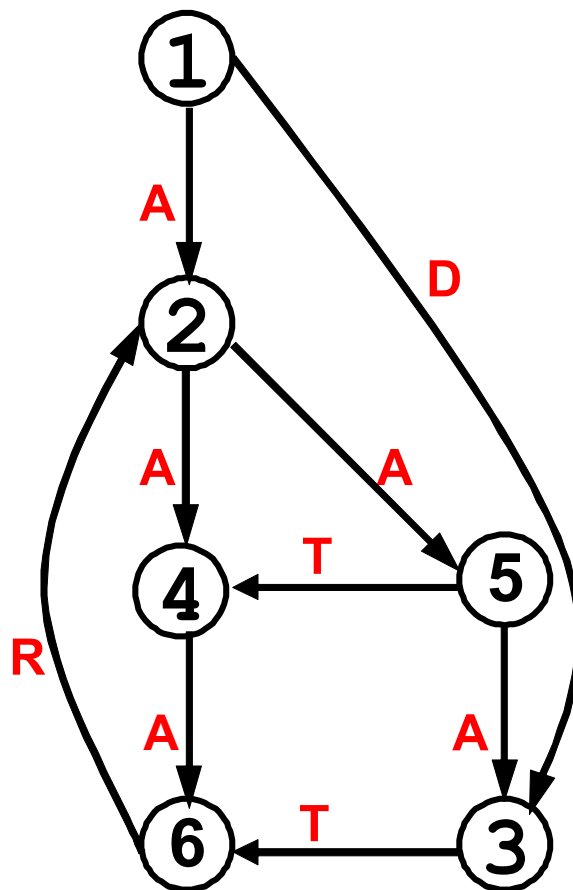
Parcurgere în adâncime

Marcaj arce:

A – arbore; **R** – revenire; **T** – traversare; **D** – direct

Descriere graf Parcurgere începând de la nodul 1

6
1 2 30
1 3 10
2 4 15
2 5 20
3 6 10
4 6 30
5 3 20
5 4 15
6 2 10



1 - 2 (A)
2 - 4 (A)
4 - 6 (A)
6 - 2 (R)
2 - 5 (A)
5 - 3 (A)
3 - 6 (T)
5 - 4 (T)
1 - 3 (D)

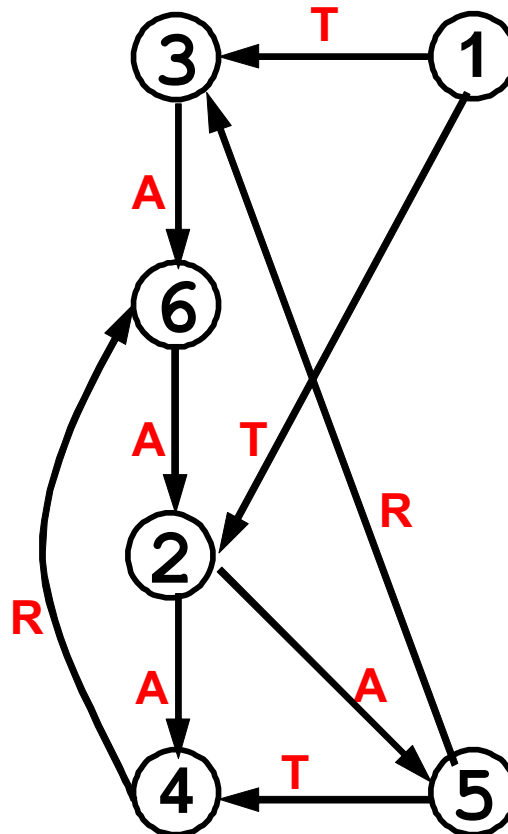
Parcurgere în adâncime

Marcaj arce:

A – arbore; **R** – revenire; **T** – traversare; **D** – direct

Descriere graf Parcurgere începând de la nodul 3

6
1 2 30
1 3 10
2 4 15
2 5 20
3 6 10
4 6 30
5 3 20
5 4 15
5 3 20
6 2 10



3 - 6 (A)

6 - 2 (A)

2 - 4 (A)

4 - 6 (R)

2 - 5 (A)

5 - 3 (R)

5 - 4 (T)

1 - 2 (T)

1 - 3 (T)

Parcurgere în adâncime

Pentru stabilirea tipului arcelor se utilizează **indicele de parcurgere $ip[i]$** , asociat nodului **i** , cu semnificatia

- 0 – netratat (alb)
- > 0 – în curs de tratare (gri)
- < 0 – tratat (negru)

$ip[0]$ = valoare indice curent (**IC**)

Cod_arc(x,a):

- A** - nodul **a** netratat ($ip[a] == 0$)
- R** - nodul **a** in_curs de tratare ($ip[a] > 0$)
- T** - nodul **a** tratat anterior ($ip[a] < 0$ si – $ip[a] < ip[x]$)
- D** - nodul **a** tratat ulterior ($ip[a] < 0$ si – $ip[a] > ip[x]$)

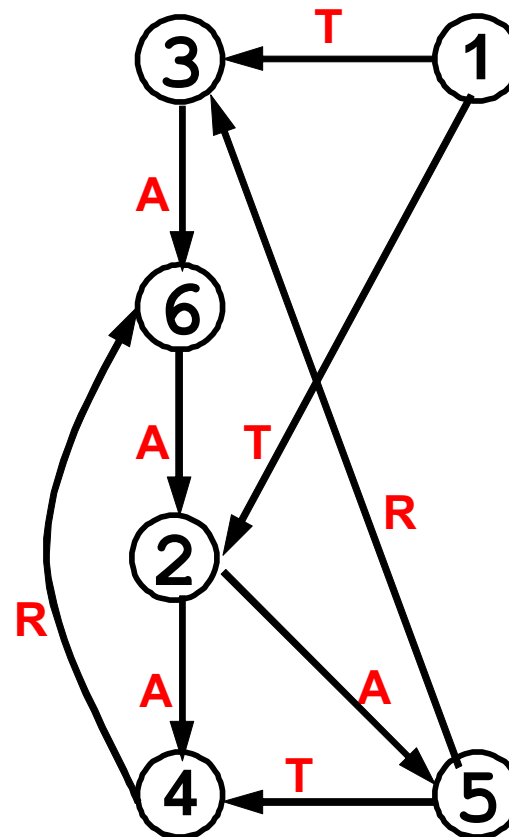
Algoritm parcurgere în adâncime

```
Algoritm ParcAd(info_parcurgere, x)
{ crește IC;
  marchează x "in_curs" (ip[x] = IC);
  pentru fiecare nod adiacent(a)
  { afișează x,a,Cod_arc(x,a)
    dacă (a "netratat") atunci
    { crește nivel;
      ParcAd(info_parcurgere, a);
      reduce nivel;
    }
  }
  marchează x ca "tratată" (schimbă semn ip[x]);
}
```

Algoritm testare graf aciclic

Descriere graf

1	2	30
1	3	10
2	4	15
2	5	20
3	6	10
4	6	30
5	3	20
5	4	15
6	2	10

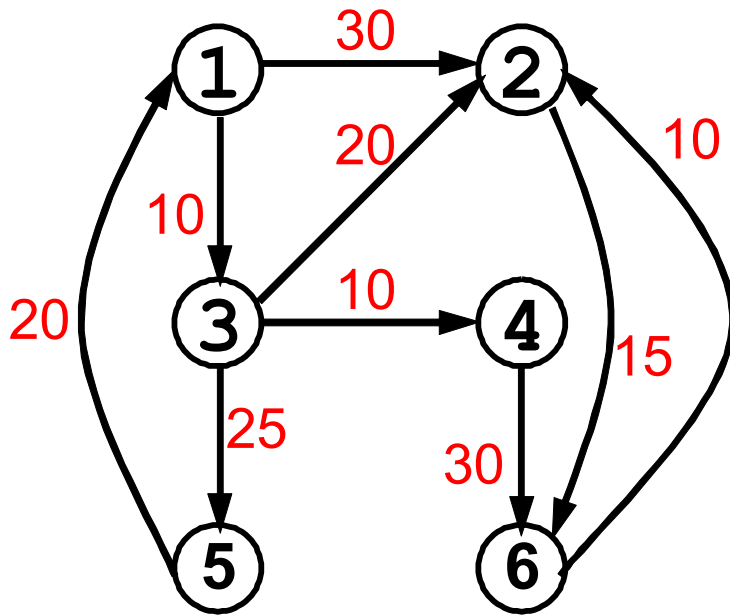


Algoritm testare graf aciclic

```
Algoritm Aciclic(info_parcurgere, x)
{ marchează x "in_curs";
  pentru fiecare nod adiacent(a)
  { dacă (a "in_curs") atunci întoarce 0;
    dacă a "netratat" atunci
      dacă (!Aciclic(info_parcurgere,a))
        atunci întoarce 0;
  }
  marchează x ca "tratată";
  întoarce 1;
}
```

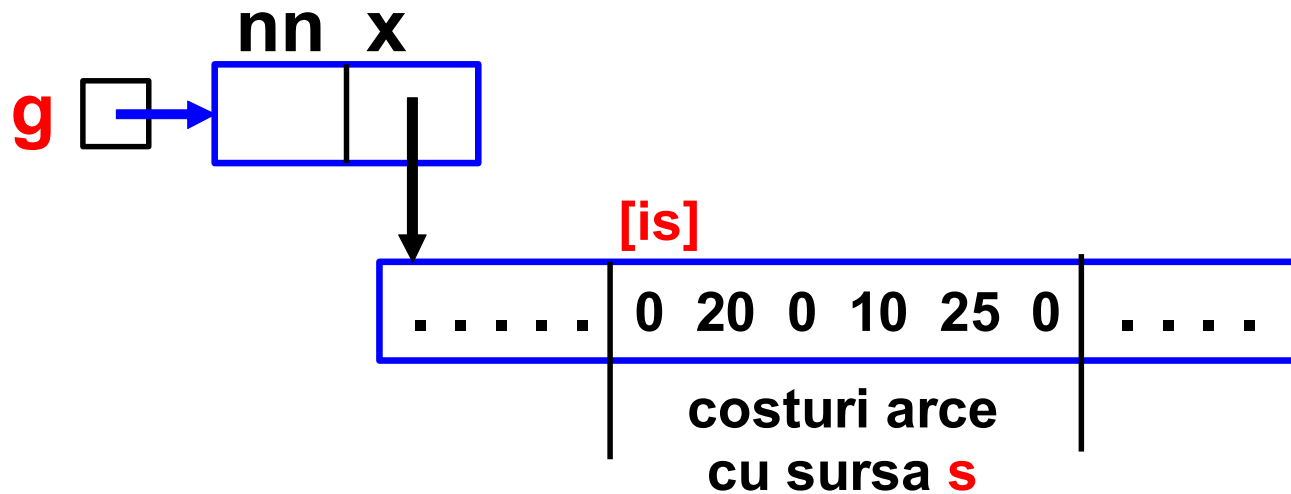
Reprezentare graf

Matrice de adiacențe (costuri)

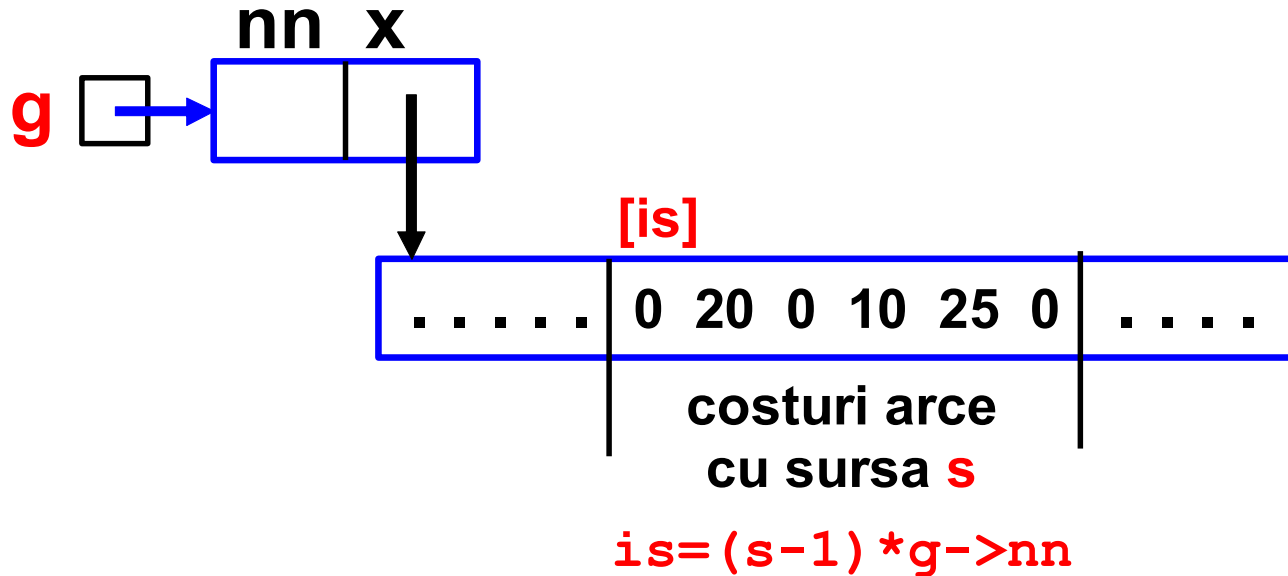


	1	2	3	4	5	6
1	-	30	10	-	-	-
2	-	-	-	-	-	15
3	-	20	-	10	25	-
4	-	-	-	-	-	30
5	20	-	-	-	-	-
6	-	10	-	-	-	-

1. Reprezentare ca matrice liniarizată costuri



Reprezentare ca matrice liniarizată costuri



```
typedef struct
{ int nn;          /* numar noduri */
  int* x;          /* vector costuri =
                    matrice liniarizata */
} TGM, *AG;        /* descriptor */
```


Reprezentare graf prin matrice de adiacență

```
typedef struct
```

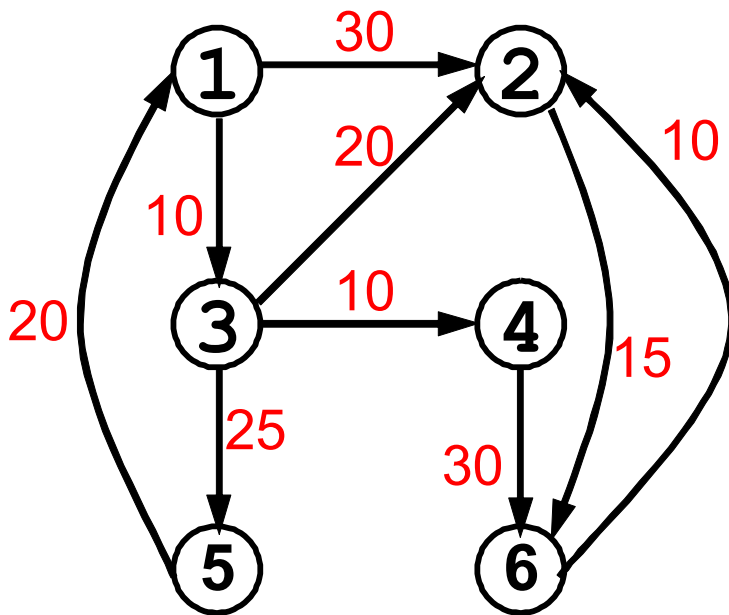
```
{ int n;    /* numar noduri */  
  int* x;   /* costuri = matrice liniarizata */  
} TGM, *AGM;
```

```
void* g; //graf
```

```
Cost arc (i, j) = ?
```

```
((AGM) g) ->x[ (i-1) * ((AGM) g) ->n + j-1] = x;
```

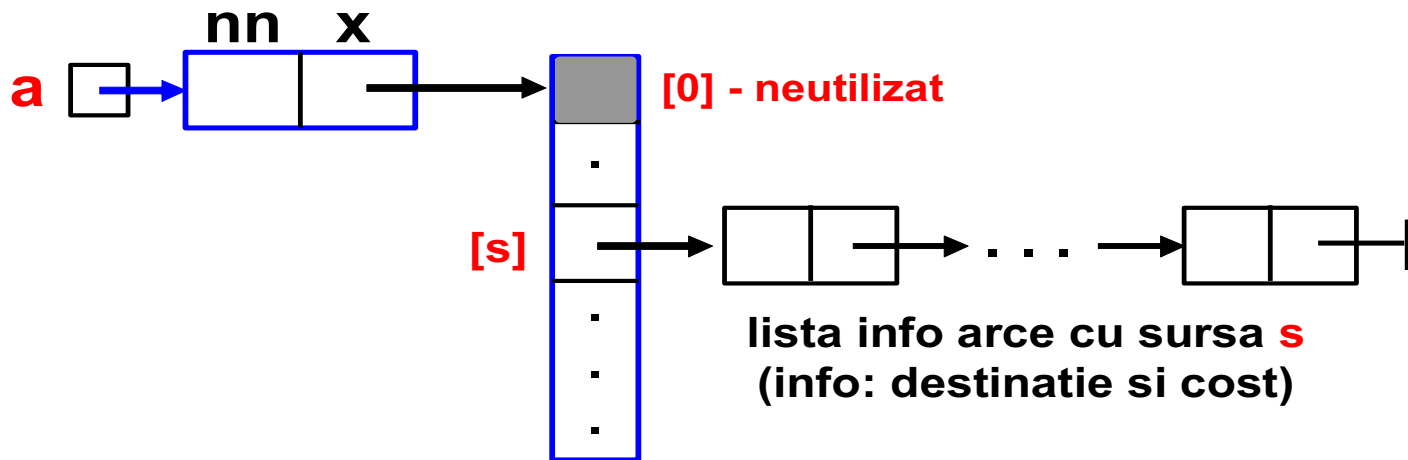
Reprezentare graf prin listă de adiacență



Liste de adiacențe

1 :	2 (30)	3 (10)	
2 :	6 (15)		
3 :	2 (20)	4 (10)	5 (25)
4 :	6 (30)		
5 :	1 (20)		
6 :	2 (10)		

Reprezentare cu liste înlănțuite de info arce



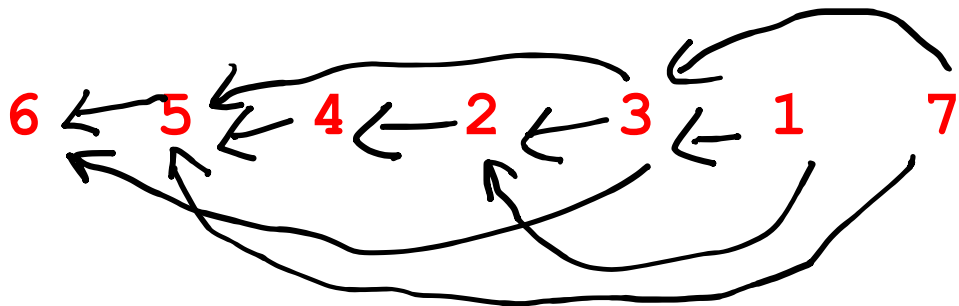
```
typedef struct celarc
{ int d;           /* destinatie arc */
  int c;           /* cost arc */
  struct celarc *urm;
} TCelArc, *AArc;

typedef struct
{ int nn;          /* numar noduri */
  AArc *x;         /* vector adrese liste arce */
} TGL, *AG;
```

Sortare topologică

	1	2	3	4	5	6	7
1	-	40	35	-	-	-	-
2	-	-	-	25	-	-	-
3	-	20	-	-	15	30	-
4	-	-	-	-	20	-	-
5	-	-	-	-	-	15	-
6	-	-	-	-	-	-	-
7	-	-	12	-	15	-	-

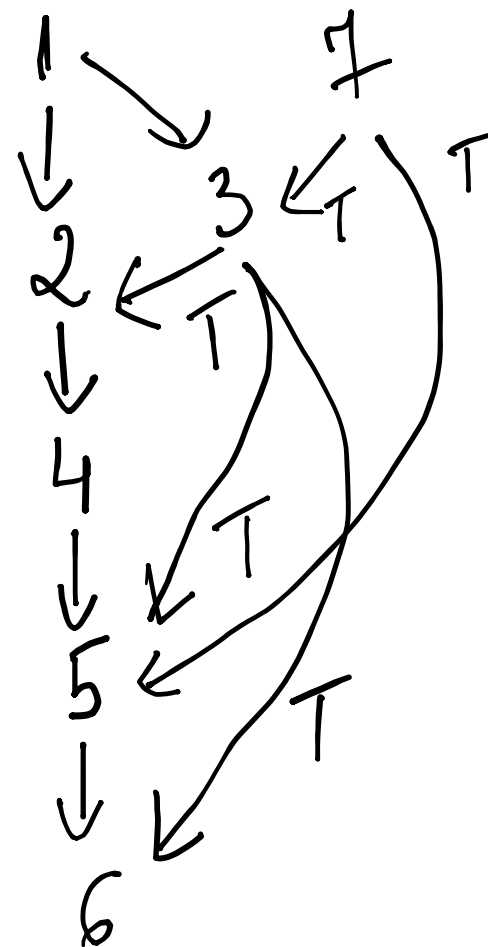
Rezultat sortare topologica:



Sortare topologică

	1	2	3	4	5	6	7
1	-	40	35	-	-	-	-
2	-	-	-	25	-	-	-
3	-	20	-	-	15	30	-
4	-	-	-	-	20	-	-
5	-	-	-	-	-	15	-
6	-	-	-	-	-	-	-
7	-	-	12	-	15	-	-

6, 5, 4, 2, 3, 1, 7



Sortare topologică

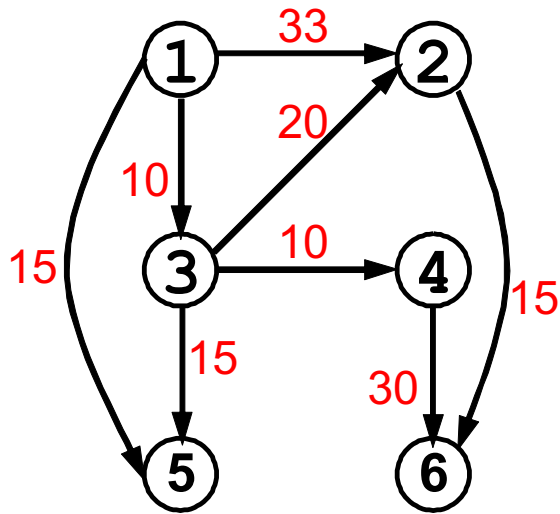
	—	1 —	2 —	3 —	4 —	5 —	6 —
1	—	—	30	10	—	—	—
2	—	—	—	—	—	—	15
3	—	—	20	—	10	25	—
4	—	—	—	—	—	—	30
5	20	—	—	—	—	—	—
6	—	—	10	—	—	—	—

Ciclu pe calea: 1 2 6 2

Algoritm sortare topologică

```
Algoritm SortTopologică(info_parcurgere, x)
{ marchează x "in_curs";
  pentru fiecare nod adiacent(a)
  { dacă (a "in_curs") atunci întoarce 0;
    dacă a "netratat" atunci
      dacă (!SortTopologică(info_parcurgere, a))
        atunci întoarce 0;
  }
  marchează x ca "tratată";
  memorează ca "sortat"
  întoarce 1;
}
```

Algoritmul Dijkstra



Sursa: 1

selectate	Distanțe				
	2	3	4	5	6
1	33	10	INF	15	INF
1, 3	30	10	20	15	INF
1, 3, 5	30	10	20	15	INF
1, 3, 5, 4	30	10	20	15	50
1, 3, 5, 4, 2	30	10	20	15	45

Algoritmul Dijkstra

Dijkstra: Distanța minimă de la sursă la celelalte noduri

```
{ initializeaza structuri de date auxiliare
  (vector distante fata de sursa,
  vector predecesori);
marcheaza sursa "tratat";
initializeaza nr = nr.noduri - 1;
cat timp nr > 0
{ alege x - nodul netratat cel mai apropiat
  de sursa;
  daca nu exista atunci iesire fortata (break);
  marcheaza nodul x ca "tratat";
  pentru fiecare nod netratat (j)
    daca adiacent lui x, inaccesibil din sursa
      sau accesibil pe un drum mai lung
        actualizeaza distanta si predecesor;
  }
}
```